Can a dosing policy do better by remembering, or by thinking in continuous time?

Continuous-time reinforcement learning for adaptive cancer therapy: what we built, how it works, what we found

Status report, 18 October 2026 virtual patients only, no clinical claims

The question, in three steps

1 Adaptive therapy

Treat until the tumour shrinks, pause, restart when it regrows. Sensitive cells stay alive and hold the resistant cells back.

2 A learned policy

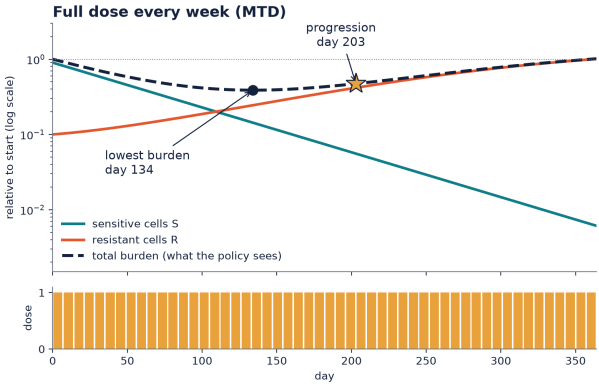
A neural network reads the tumour measurement each week and picks a dose: none, half or full. It learns by trial and error on virtual patients.

3 What kind of network?

Does a network that remembers the past (LSTM) or models time as a flow (ODE-RNN) beat one that sees only today's value (MLP)?

Answer in our simulator: no. The memoryless network matches or beats both. The ODE-RNN is about 50 days worse, and the gap is a training cost, not a limit of what the network can represent.

Why full dosing stops working



1 Start. Mostly sensitive cells S, a few resistant cells R, competing for the same space.

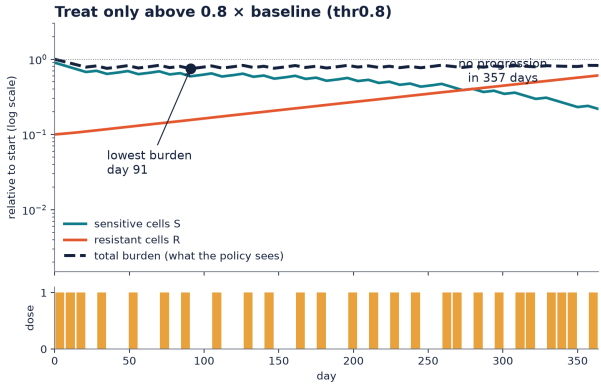
2 Full dose. The drug stops S growing, so S shrinks and total burden falls.

3 Release. With S gone, nothing holds R back. The drug does not touch R, so R grows into the space.

4 Regrowth. Burden follows R. The dose is still maximal, but there is nothing left to act on.

Illustrative patient from a re-implementation of the paper's equations ($r_R/r_S = 0.9$, $d_T/r_S = 0.5$, start fill 0.3, resistant fraction 0.1; inside the Strobl ranges). Not the patient of the paper's Figure 1.

Sparing the sensitive cells delays the regrowth



203 d

MTD: progression

>357 d

thr0.8: none

52

MTD: doses

26

thr0.8: doses

thr0.8 doses only while burden is above 0.8× its start. Some S survives and R is 74% of the tumour at the end, against 99% under MTD. Burden stays higher for much of the year: the price of a later regrowth.

The virtual patient: two populations competing

$$\frac{dS}{dt} = r_S S\left(1 - \frac{S+R}{K}\right)(1 - d_D u) - d_S S$$

$$\frac{dR}{dt} = r_R R\left(1 - \frac{S+R}{K}\right) - d_R R$$

Patients are Latin-hypercube samples of the Strobl et al. (2021) ranges: r_R/r_S 0.5 to 1, turnover 0 to 0.5 r_S, start fill 0.1 to 0.75 of K, resistant fraction 0.1% to 10%. With those parameters our code reproduces all 14 published progression times and gains we checked, within 3%.

The policy never sees S, R or the parameters: only total burden, with noise.

S, R sensitive and resistant cells

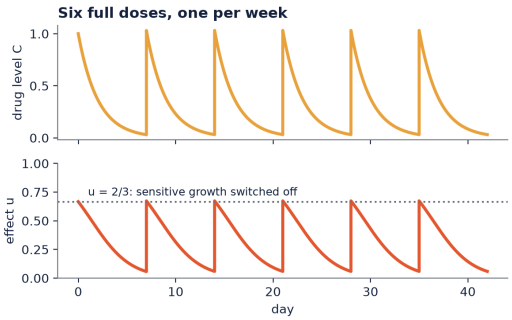
r_S, r_R growth rates; $r_R < r_S$ is the cost of resistance

K shared capacity

d_S, d_R natural death (turnover)

$d_D u$ drug kill, on S only

The drug: a bolus that decays



$$u = \frac{C}{C + EC_{50}}, \qquad EC_{50} = 0.5$$

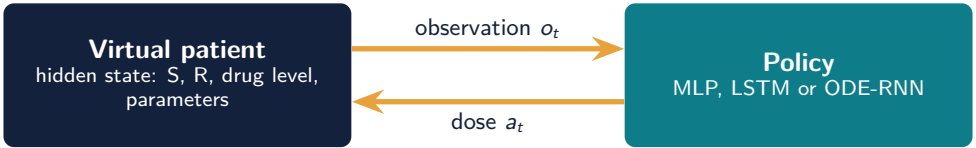
$$C \leftarrow C + a_t \text{ at a visit}, \qquad C \leftarrow Ce^{-0.5\,\Delta t}$$

A dose is added to one compartment at each weekly visit and cleared at 0.5 per day.

The effect saturates: doubling the dose does not double the kill.

A full dose gives $u \approx 2/3$, where $1 - d_D u$ reaches 0: sensitive growth is switched off and S declines by turnover.

The decision problem is a POMDP



Each week the policy gets o_t, a noisy log of total burden, and picks a dose a_t; the patient then evolves for 7 days. 52 weeks per episode.

Why partially observable? The same burden can come from very different mixes of S and R. A falling burden can mean the sensitive cells are dying while resistant ones already dominate.

Why memory could help. The history of burden carries information about the hidden mix. That is the case for the LSTM and ODE-RNN, and what we test.

Observation, reward and score

Sees $o_t = \log \dfrac{S_t + R_t}{S_0 + R_0} + \varepsilon_t, \quad \varepsilon_t \sim \mathcal{N}(0, 0.05^2)$. Never S, R or the parameters. A missed visit gives no observation; the last dose continues.

Is rewarded for $r_t = -\max\left(\dfrac{b_t}{b_{\min,t}} - 1.2,\ 0\right) - w\, a_t$ (+const.), with $b_t = S_t + R_t$ and $w \in \{0.05, 0.1, 0.2, 0.4\}$; one network per w.

Is reported as $U = \min(\mathrm{TTP}, 357) - w \sum_{t=1}^{52} a_t$. Progression: burden 20% above its lowest, for 28 days.

Optimises $\max_\pi \ \mathbb{E}\Big[\sum_{t=1}^{52} \gamma^t r_t\Big]$, $\gamma = 0.99$, $a_t \sim \pi(\cdot \mid o_{1:t})$, $a_t \in \{0, \frac{1}{2}, 1\}$.

The reward is a proxy for the score: it penalises burden at every visit, while the score counts days to a confirmed progression.

The four networks compared

MLP

Sees only today's measurement. The memoryless baseline.

Positional MLP

Today's measurement plus a clock. Tests whether knowing the time is enough.

LSTM

Recurrent: carries a summary of the past in discrete steps. The standard memory.

ODE-RNN

Hidden memory flows as a differential equation between visits. The continuous-time candidate.

Fair comparison. Same trainer (PQN, a simple Q-learning method), same size (about 3,100 parameters), published settings, no per-network tuning, 10 seeds each. Only the architecture differs.

How we test: three robustness settings

Tier 0: in range

Test patients come from the same ranges as training.

Tier 1: shifted

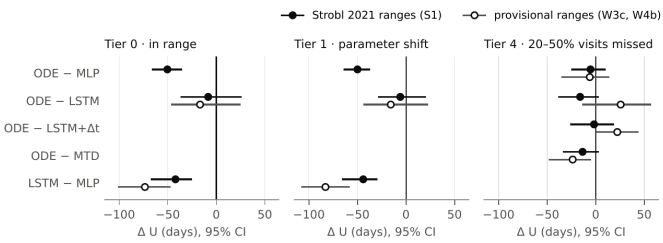
Growth rates $\times 1.5$ and capacity $\times 0.6$: outside the training ranges.

Tier 4: missed visits

Each visit missed with probability 20 to 50%; the last dose continues.

200 + 200 training and test patients per seed **10** seeds per network **95% CI** paired bootstrap

Result 1: no continuous-time edge



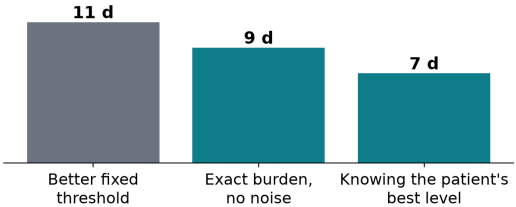
Dot: mean score difference (days); line: 95% interval; right of zero favours the first-named network.

ODE-RNN minus MLP: −50.0 d [−63.5, −37.9] under the shift.

ODE-RNN minus LSTM: −5.9 [−27.9, +19.9], no difference.

Missed visits: −5.4 [−24.3, +9.3]; never above a fixed-schedule MTD rule.

Headroom exists; little of it needs memory



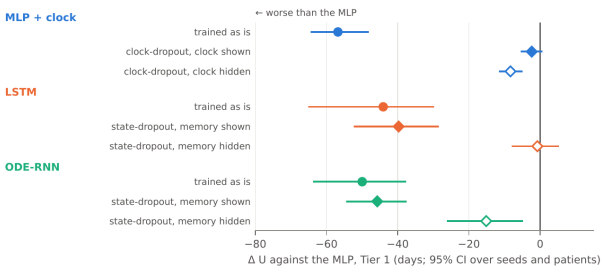
Gain in days of a search on the true simulator, split by what it needs (approximate thirds).

+23.1 d

search on the true simulator over the MLP [+20.3, +25.8]

Only the last two bars need memory-type information: at most about 15 days. Training the recurrent networks costs 40 to 50 days. This is a split of one rule family's gain, not a proof that no memory policy could help.

Result 2: a training shortcut, found and removed



The clock costs the MLP 57 days. The trainer fits a time-keyed schedule instead of reacting to the tumour.

Hide the clock half of the time and the cost goes: +54.4 d, tied with the MLP (−2.4 [−5.2, +0.4]).

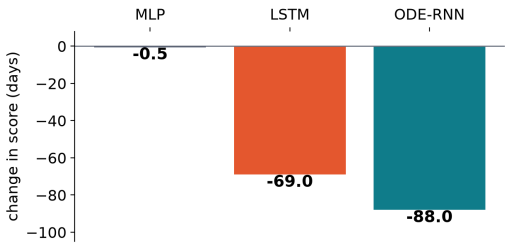
The same trick on LSTM and ODE-RNN changes little (+4.3, +4.2 d); both stay 40 to 46 d below the MLP.

Objections we tested

Objection	Run	Result
The trainer cannot use memory	W10: three POPGym memory tasks	LSTM beats the MLP on 2 of 3: the trainer can use memory where it pays
The LSTM is too small	V11: 5× and 22× the parameters	No recovery: still 47 to 49 d below the MLP
Only one training algorithm	V12: PPO instead of PQN	Inconclusive: validity check failed; recurrent arms collapse to a fixed schedule in 12 and 22 of 40 runs
They cannot represent the policy	W11: clone a threshold policy, then fine-tune	All three can; PQN fine-tuning destroys it in the recurrent arms only

Each run was registered, with pass and kill criteria, before it was run.

A learning cost, not a representation limit



Change in score when PQN fine-tunes a network that already copies a good threshold policy.

Step 1. Teach each network to copy a threshold rule by supervised learning. All three succeed (score about 284.6 d).

Step 2. Let PQN keep training. The MLP is unchanged; the LSTM loses 69 days and the ODE-RNN 88.

So: the recurrent networks can hold the good policy, but PQN's training moves them away from it. Only PQN was tested.

Limits, and what comes next

Limits

One simulator family (Lotka-Volterra, competitive resistance); virtual patients; no clinical claims.

Mostly one training algorithm, untuned; recurrent networks are unstable across seeds.

Weekly visits and fixed 7/8-day ODE-RNN steps: missed visits carry no timing information.

Next

A second training algorithm that passes the full-information check; it needs a new idea, not a re-tune.

Optional: a tighter bound on the headroom, and an MLP scaled up to match.

Anonymised code link, then submission to TMLR.